

Name: Sam Setta
Date: June 3, 2026
Course: Foundations Of Python Programming
Assignment: Assignment 07
Github URL:

Abstraction, Encapsulation, and Inheritance

Introduction

Here, I outline how to add programming concepts of abstraction and encapsulation to the student registration from Assignment 06. Two data classes were added to the program, one super class for a person (first and last name) and one sub class for students which adds course name to the students name. The student enrollment data from the Enrollment.csv file was then converted from a dictionary to a two-dimensional list to work with the student class. Program testing included troubleshooting a few errors after adding the new data classes, and making sure the same functionality as the Assignment 06 program remained. Unlike previous assignments, here I will only outline the program additions.

Adding data classes

Data classes were used to organize data about each student using data abstraction and encapsulation with getters and setters in Python.

Person data class

The 'Person' data class was defined within the constructor (`__init__` method) with the attributes of first name and last name, and the self keyword to reference the instance of the 'Person' class. To include data validation within the Person class, following best practices for data abstraction and encapsulation, private attributes were used with the property. The getter `@property` decorator used in the code accesses the `first_name` attribute and applies formatting of title case and as a private attribute, before the setter property function is used. The setter allows data validation for the student first_names by calling the attribute with the decorator (`@first_name.setter`), where the name is checked to ensure there are no numbers accidentally included. If numbers are included in the name the setter raises a custom error that names must only include alphabetic entries. The same data validation from the student's first name entry is done with the student's last name using the getter and setter property functions (see code below).

```
# Add first_name properties to the constructor
@property
```

```

def first_name(self):
    return self.__first_name.title() # formatting code

@first_name.setter
def first_name(self, value: str):
    if value.isalpha() or value == "":
        self.__first_name = value
    else:
        raise ValueError("The first name should not contain numbers.")

# Add last_name properties to the constructor
@property
def last_name(self):
    return self.__last_name.title() # formatting code

@last_name.setter
def last_name(self, value: str):
    if value.isalpha() or value == "":
        self.__last_name = value
    else:
        raise ValueError("The last name should not contain numbers.")

```

The output from the Person data class is changed to the student's first and last name separated by a comma by changing the 'to string' method (`__str__`) to return the Person attributes.

Student data class

The Student class is an inherited class from the parent Person class (e.g. `class Student(Person)`) and could 'inherit' the first and last name from Person by using the super method with the constructor (`super().__init__`) with the first_name and last_name defined attributes. The course_name attribute was added to the student class, and the getter and setter property methods were used to format the course_name and validate the course_name is a string. The Student class 'to string' output was also customized to output the attributes from both the inherited class (first_name and last_name) and the course_name.

Adapting functions to work with data classes

Converting a dictionary from a JSON file into list of lists

Because the student class data is now a list of lists, the data from Enrollment.json needs to be converted from a dictionary to a two-dimensional list. This was done within the read_data_from_file method in the FileProcessor class. An empty list was set to

student_objects, after which each row from the loaded JSON file was iterated through using a for loop. Each 'row' or entry into the JSON file dictionary was saved to the corresponding Student class attribute using the keys from the dictionary within the Student data class (see below). The data from the row was converted to an instance of the Student class was then appended to the student_objects list, and the student_objects is returned at the end of the method.

```
# Convert the list of dictionary rows into a list of Student objects
student_objects = []
# replace this line of code to convert dictionary data to Student data
for row in json_students:
    student_obj = Student(first_name=row["FirstName"],
                          last_name=row["LastName"],
                          course_name=row["CourseName"])
    student_objects.append(student_obj)
```

Converting student data to a dictionary to save as a JSON file

When saving the student_data to a JSON file, the data needed to be converted from a list back into a dictionary, this was done within the write_data_to_file method in the FileProcessor class. The inverse of read_data_from_file method was needed, with an empty dictionary (json_students) defined, before using a for loop to convert each list within the student_data list into a dictionary by matching student class attributes with associated keys (e.g. 'FirstName', 'LastName', 'CourseName'), and appending each key-value formatted 'row' to the json_students dictionary. The json_students dictionary can then be used with the json.dump() function to write the data to the Enrollment.csv file.

Editing IO methods to access Student object data

Some of the remaining methods were edited to reference the two dimensional list rather than the dictionary used in Assginment_06.py.

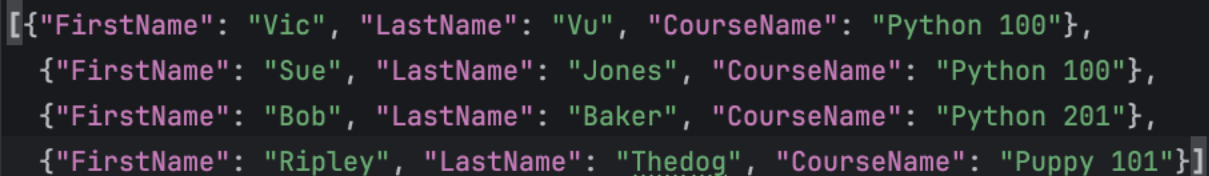
The IO.output_student_and_course_names() method had to be changed to access the Student data object rather than the dictionary by using a for loop to iterate through each student data list (student) and printing the attributes of the class separated by a comma (*print(student.first_name, student.last_name, student.course_name)*). The IO.input_student_data() method was changed to add new entries to the student class attributes rather than dictionaries by defining a new entry as 'new_student' and setting that equal to student class data with the attributes given the input values as arguments, before appending the new student list to the student_data list of lists.

Testing the program

Testing the program in PyCharm

While writing this script, I encountered more errors than in previous assignments due to the new data classes and dictionary conversions from the JSON file. The first error when testing in PyCharm was an output of first_name, first_name, course_name. After using debug and working through several methods, include the function to read data from the JSON file, I discovered the bug was in the initially defined Person class, for which the last_name.setter() was referencing the first_name.setter() instead (an error we were warned about in the notes!), once this was fixed, the program ran as expected. The second bug kept producing the Student data object memory location (default of __str__) instead of the customized output, after debugging, I discovered this was due to an indentation error.

After both of these bugs were fixed, the program ran as expected, accepting multiple entries (Bob Baker Python 201, and Ripley Thedog Puppy 101) in both the command line and PyCharm before saving to the JSON data file. I also checked that the file contained the saved data (Figure 1), before re-running the program and choosing menu option 2 to confirm that the new data was read into the program properly.



```
[{"FirstName": "Vic", "LastName": "Vu", "CourseName": "Python 100"},  
 {"FirstName": "Sue", "LastName": "Jones", "CourseName": "Python 100"},  
 {"FirstName": "Bob", "LastName": "Baker", "CourseName": "Python 201"},  
 {"FirstName": "Ripley", "LastName": "Thedog", "CourseName": "Puppy 101"}]
```

Figure 1. Screenshot of the data file after entering two new student registrations.

Summary

In this document, I've outlined how to use data abstraction and encapsulation to define a 'Person' data class and a 'Student' sub class that adds the course name information to the inherited student first and last name from Person. Working with data classes as defined in this program required the conversion of dictionaries from the JSON data file to two-dimensional lists and vice-versa to save the student data to a JSON file. This program had a few bugs that were discovered during testing, which were isolated and fixed with the use of the PyCharm debugger.